Synchronous Serial Interfaces: SPI and I^2C

Introduction to ARM Cortex-M Microprocessors — Chapter 13

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1 Why This Matters

2 The SPI Bus

3 I^2C

Why This Chapter Matters

UART has no clock wire, so both ends must agree on the baud rate. Adding a clock wire removes that need and the speed limit. SPI is fast and simple, but costs one pin per device. I^2C is slower, yet it connects dozens of devices on two wires. You will read a real sensor and turn its raw values into a physical measurement.

After this chapter, you can:

1. Name the four SPI signals; explain full-duplex exchange.
2. Choose CPOL and CPHA; calculate f_{SCK}.
3. Configure an SPI master and do a register transaction.
4. Describe the I^2C phases; explain open-drain and pull-ups.
5. Configure an I^2C master and read a slave register.
6. Choose between SPI and I^2C, and justify it.
7. Convert a real sensor's output to engineering units.

© Muním Zabidi

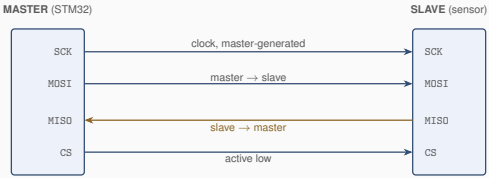
Table of Contents

1 Why This Matters

2 The SPI Bus

3 I^2C

Four Wires, Full Duplex

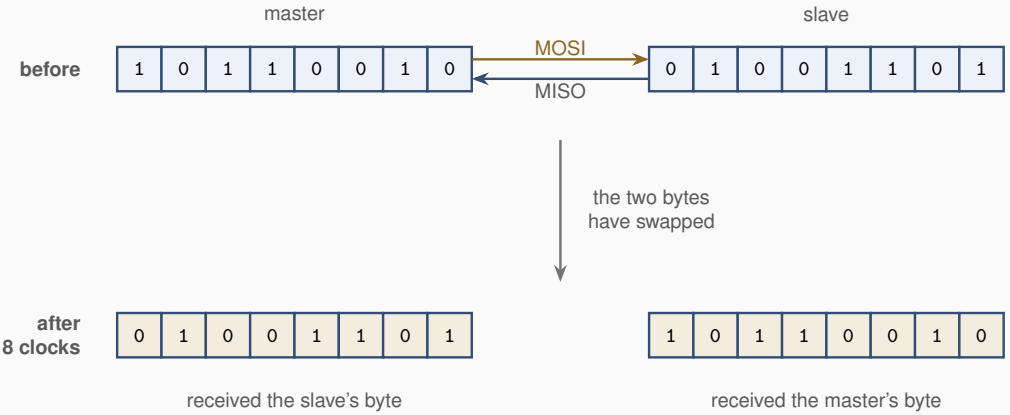


Signal	Direction
SCK	Master → slave
MOSI	Master → slave
MISO	Slave → master
CS	Master → slave, per device

A transfer is an *exchange*, not a send

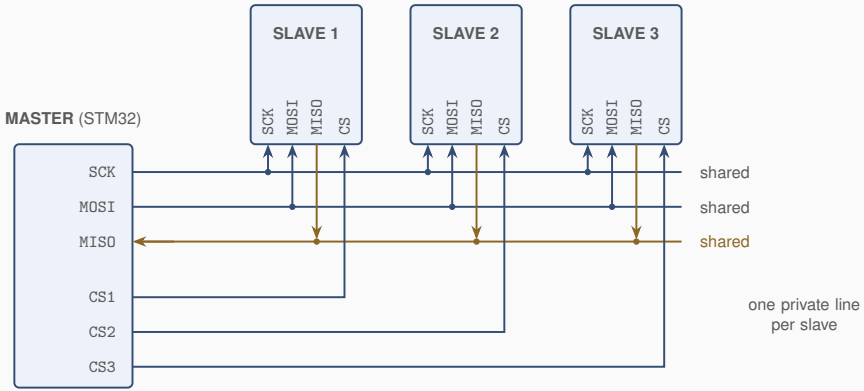
Master and slave are one circular shift register. Every clock edge shifts a bit out of each and into the other — so sending a byte *always* receives one at the same time. To read a device you must still write something (often a dummy byte) to generate the clock.

Full Duplex by Shift Register



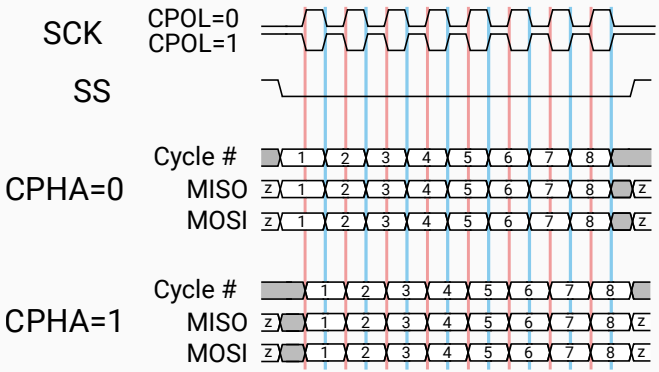
SPI is an exchange, not a send or a receive. The two shift registers form one ring, so eight clock pulses trade their contents.

More Than One Slave



`SCK`, `MOSI`, and `MISO` are shared by every device; only chip select is private. Adding a slave costs exactly one pin.

Clock Polarity and Phase



CPOL sets the idle level of the clock; **CPHA** sets which edge samples the data. Four combinations, called modes 0 to 3 — and the datasheet tells you which one your device needs.

Configuring an SPI Master

```
RCC->APB1ENR |= (1U « 14); // SPI2

SPI2->CR1 = (1U « 2)       // MSTR
          | (3U « 3)       // BR: /16
          | (1U « 9)       // SSM
          | (1U « 8);      // SSI

SPI2->CR1 |= (1U « 6);     // SPE
```

$$f_{SCK} = \frac{f_{PCLK}}{2^{(BR+1)}}$$

With BR $= 3$ and a 16 MHz peripheral clock, $f_{SCK} = 1$ MHz.

Why this course uses SPI2, not SPI1

SPI1's default SCK is PA5 — the user LED. Using SPI1 would silently break every blink program in the book. SPI2 sits on **APB1** (not APB2), and its pins PB13–PB15 live in AFR[1], indexed as $(pin - 8) \times 4$.

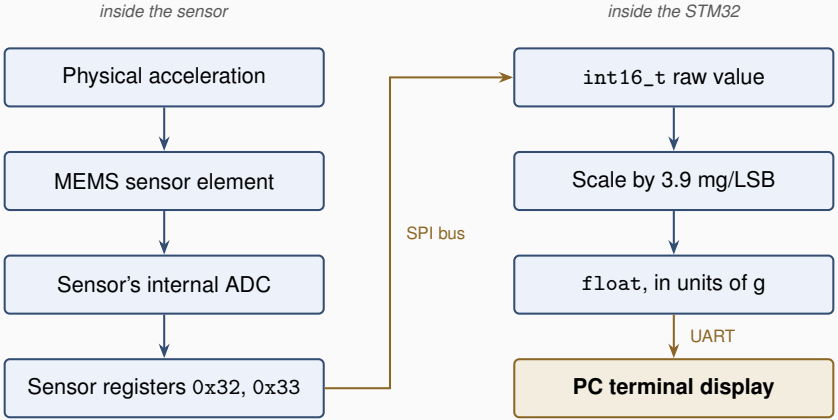
A Register Transaction

```
uint8_t spi_transfer(uint8_t out)
{
    while (!(SPI2->SR & (1U << 1))) ;    // TXE -- ready to send
    *(volatile uint8_t *)&SPI2->DR = out;
    while (!(SPI2->SR & (1U << 0))) ;    // RXNE -- a byte came back
    return *(volatile uint8_t *)&SPI2->DR;
}

uint8_t read_register(uint8_t addr)
{
    cs_low();
    spi_transfer(addr | 0x80);       // bit 7 set = read
    uint8_t v = spi_transfer(0x00);  // dummy byte to clock the reply out
    cs_high();
    return v;
}
```

The dummy byte is the point: you cannot receive without transmitting, because the master supplies the clock.

The Complete Measurement Chain



From a physical acceleration to a number on a screen. Every stage is a topic from an earlier chapter.

Table of Contents

1 Why This Matters

2 The SPI Bus

3 I^2C

Two Wires, Many Devices

- `SCL` clock and `SDA` data — two wires total, regardless of device count
- Each device has a 7-bit address, so no chip-select lines
- Both lines are open-drain, with external pull-ups

Why open-drain is mandatory

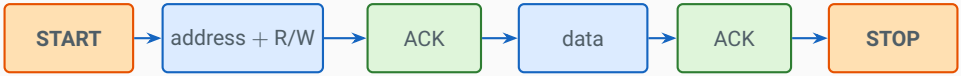
Many devices share both wires. If two drove push-pull outputs in opposite directions the result would be a short circuit.

Open-drain means a device can only *pull low* or *release* — so the pull-up defines the idle state and no contention is possible.

That is Chapter 8's open-drain output, now with a reason. The pull-up value follows from bus capacitance: too large and the rising edge is too slow; too small and it wastes current.

© Muním Zabidi

A Transaction

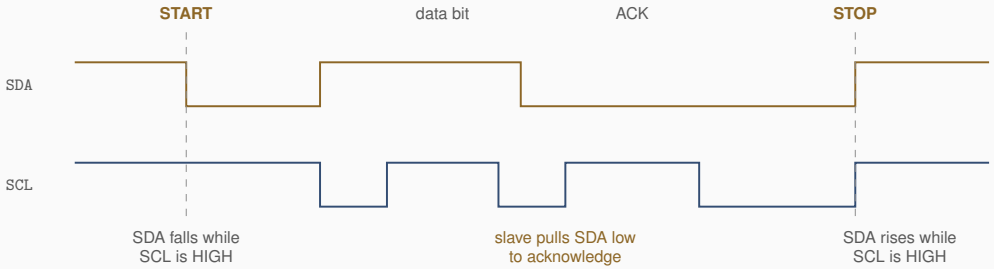


Phase	What happens
START	SDA falls while SCL is high — the one pattern that is otherwise illegal
Address	7 bits plus a read/write bit. Every device compares it against its own
ACK	The addressed device pulls SDA low. Silence means nobody answered
STOP	SDA rises while SCL is high

A missing ACK is the most common I^2C fault — and it usually means a wrong address or missing pull-ups.

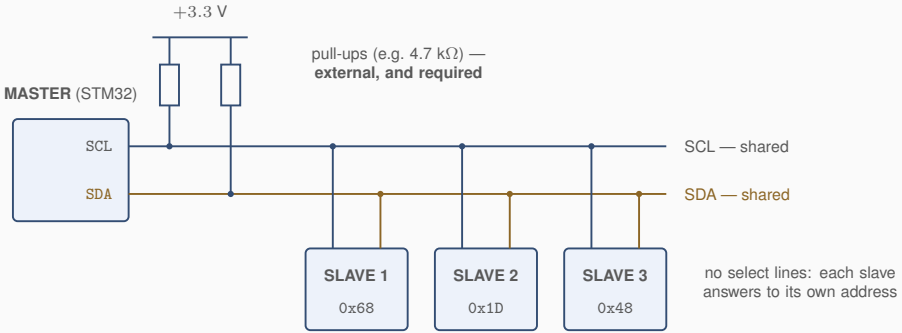
© Muním Zabidi

The Framing Conditions, on the Wire



SDA may only change while SCL is LOW, so the two transitions that occur while SCL is HIGH are unambiguous markers for the start and end of a transfer.

Many Devices on Two Wires



Compare this with SPI's three private chip-select lines: here the pin count stays at two, no matter how many devices hang off the bus — selection has moved into software.

Choosing Between Them

	SPI	I^2C
Wires	4, plus one CS per device	2, regardless of count
Speed	Tens of MHz	100 kHz to 1 MHz
Duplex	Full	Half
Addressing	A chip-select line	A 7-bit address
Acknowledge	None — you cannot tell if it worked	ACK per byte
Best for	Displays, flash, fast ADCs	Many slow sensors, EEPROM

The trade in one sentence

SPI buys speed with pins; I^2C buys pins with speed — and throws in an acknowledgement, so you know whether the device is actually there.

© Munim Zabidi

An SPI Byte at Five Levels

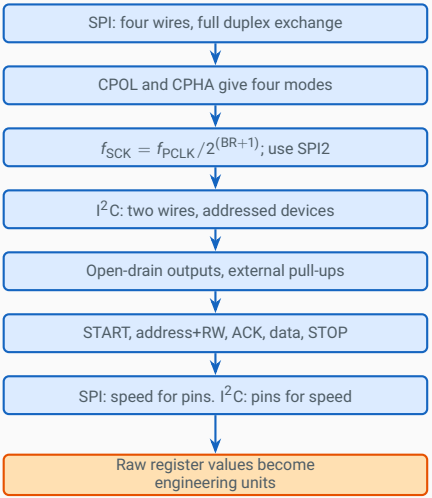
Level	Representation
Physical	Eight clock pulses on PB13; data levels on PB15 and PB14
Hardware	Two shift registers exchanging contents in a ring
Register	`SPI2->DR` written, then read
C	`value = spi_transfer(0x00);`
Engineering	One axis of acceleration, in units of g

You have now met every peripheral this book teaches

Digital pins, interrupts, timers, analog input, and three serial buses: one asynchronous and two synchronous. Chapter 14 asks you to use them all at once, with no OS to arbitrate.

© Muním Zabidi

Summary



Terminology covered

- MOSI, MISO, SCK, CS
- CPOL, CPHA
- Open-drain, pull-up
- START, ACK, STOP

Next: Chapter 14

Designing, Building and Verifying

Every peripheral so far, in one program, at once.

© Munim Zabidi